

UNIVERSIDAD MARIANO GALVEZ DE GUATEMALA



INGENIERÍA EN SISTEMAS

Manual de Usuario: Calculadora Básica

https://github.com/Elmer-Fuentes/Calculadora_Basica-Compiladores.git

DOCENTE: ING. WYLIAN ISAAC CABEZAS DURÁN

David Estuardo Cruz García

7490 23 22484

Elmer Leonel de la Cruz Fuentes

7490 23 23907

Edgar Josué López Arévalo

7490 23 5979

Cuilapa, sábado 23 de mayo de 2026

Contenido

Manual de Usuario: Calculadora Básica	1
1. Introducción.....	3
Descripción de la Conectividad de Archivos.....	3
2. Preparación del Entorno	3
3. Guía de Ejecución (Paso a Paso)	5
4. Obtención de Resultados.....	8
5. Resolución de Errores	9

1. Introducción

Esta aplicación es un analizador léxico y sintáctico desarrollado en Java. Su función es procesar expresiones aritméticas contenidas en un archivo de texto, validarlas y resolverlas respetando la jerarquía de operaciones (paréntesis, multiplicaciones/sumas/restas).

Descripción de la Conectividad de Archivos

El proyecto sigue una arquitectura de tres capas donde los archivos interactúan para transformar texto plano en resultados matemáticos:

1. Capa de Definición (Lexer.flex y Token.java):

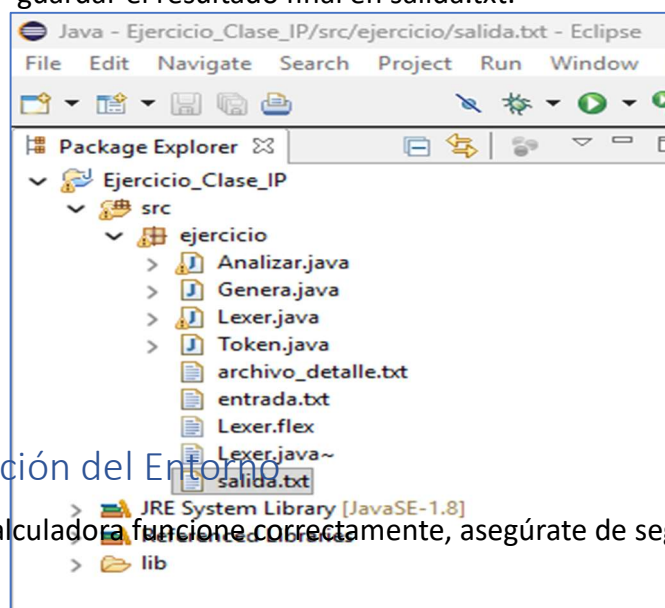
- Lexer.flex contiene las reglas léxicas (expresiones regulares) para identificar números y operadores.
- Token.java actúa como el diccionario que clasifica dichos elementos (ej. SUMA, NUMERO).

2. Capa de Generación (Genera.java):

- Este archivo compila la especificación de Lexer.flex para producir el archivo ejecutable **Lexer.java**, que es el motor que realmente "lee" el texto.

3. Capa de Procesamiento (Analizar.java):

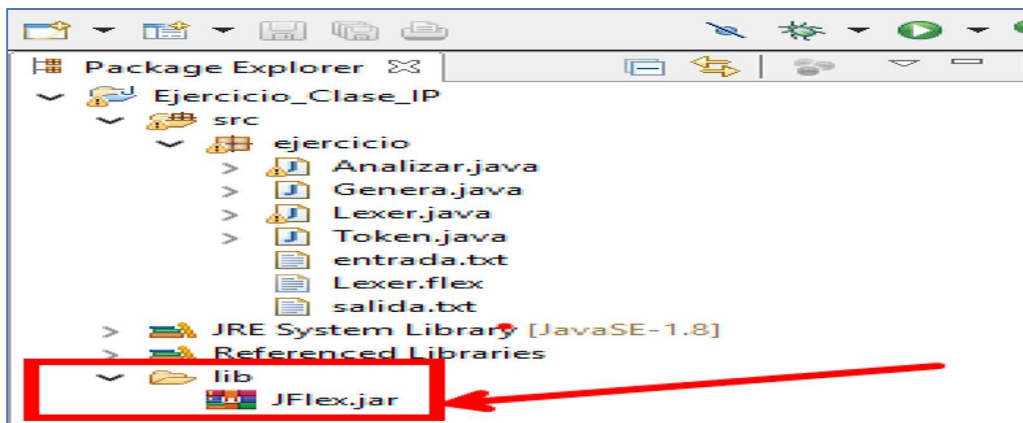
- Es el núcleo del proyecto. Lee entrada.txt y utiliza Lexer.java para convertir las líneas en una lista de tokens. Luego, aplica la lógica matemática (métodos calcular y operar) para resolver las expresiones y guardar el resultado final en salida.txt.



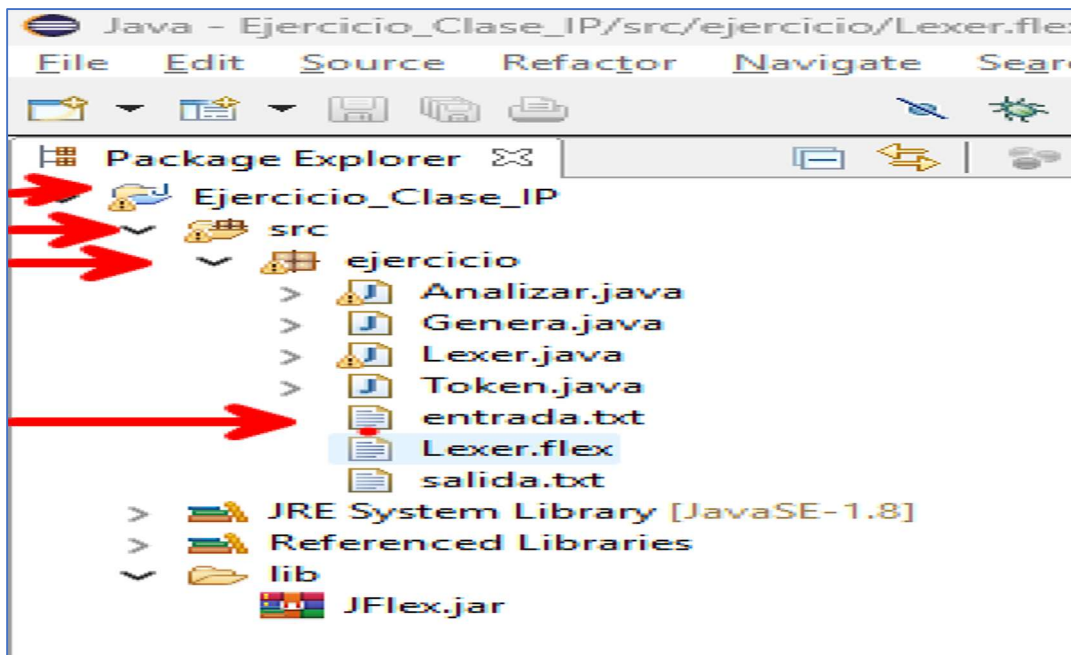
2. Preparación del Entorno

Para que la calculadora funcione correctamente, asegúrate de seguir estos pasos:

1. Instalar SDK JAVA, abrir Ide Eclipse, el proyecto esta con java 8.1 y librería de jflex.



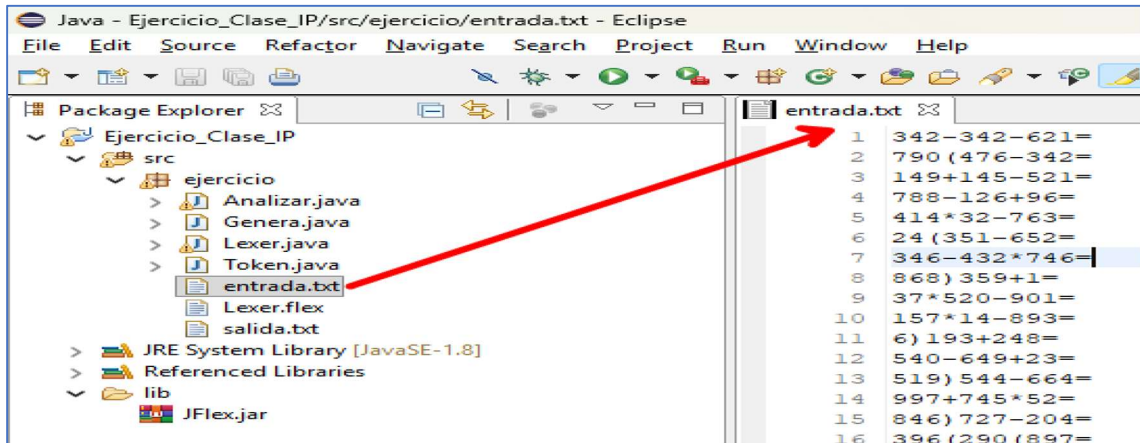
2. **Archivo de Entrada:** Ubica el archivo entrada.txt dentro de la carpeta src/ejercicio/.



3. **Formato de Operaciones:** Escribe cada operación en una línea nueva. Ejemplo:

$10 + 5 * 2 =$

$(10 + 5) / 3 =$

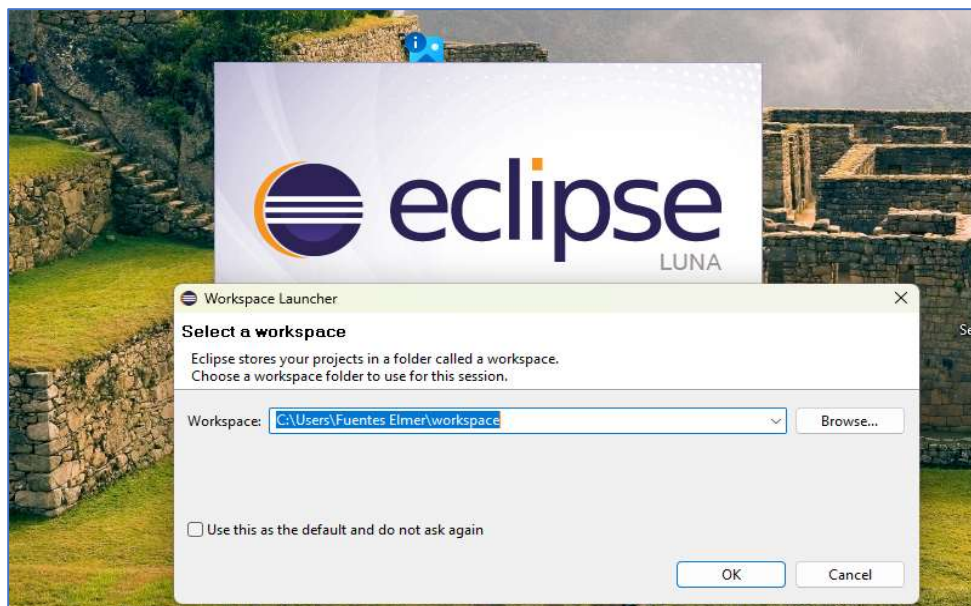


El sistema agregará automáticamente el signo = si olvidas ponerlo). Esto gracias a las expresiones léxicas.

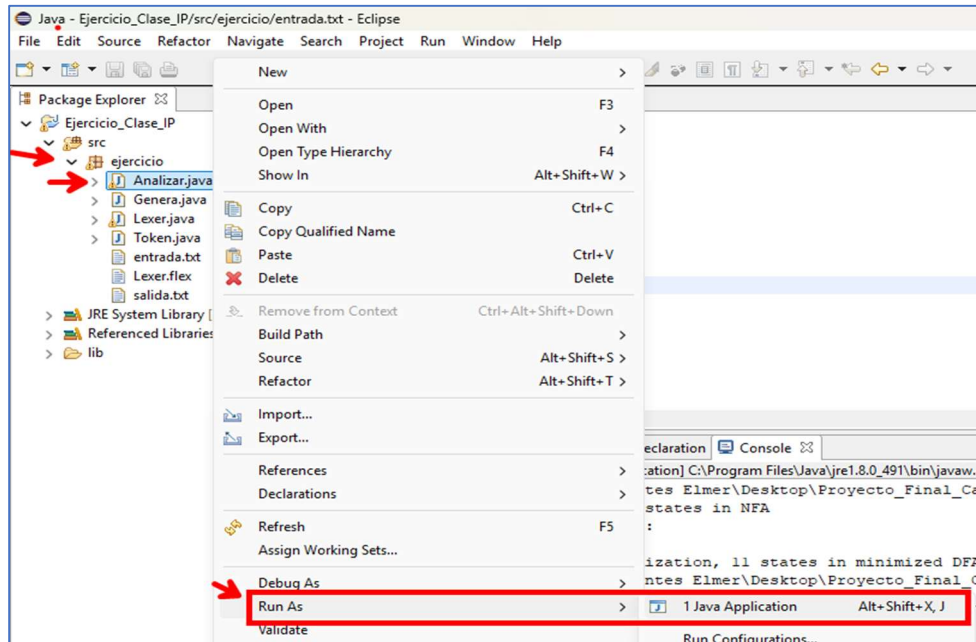
3. Guía de Ejecución (Paso a Paso)

Paso 1: Iniciar el programa

1. Abre tu IDE (Eclipse). Aplica si no lo tienes abierto.



2. Localiza la clase **Analizar.java** en el paquete ejercicio.
3. Haz clic derecho sobre **Analizar.java** → **Run As** → **1 Java Application**.



Paso 2: Selección del Modo de Detalle

Al ejecutar el programa, verás un menú en la consola. Selecciona la opción que prefieras escribiendo la letra correspondiente:

- **S (Consola):** Verás el proceso de análisis línea por línea en tu pantalla.

```
Analizar [Java Application] C:\Program Files\Java\jre1.8.0_491\bin\javaw.exe (2)
Seleccione modo de detalle:
S = Muestra operaciones en consola
N = No mostrar detalle, maxima velocidad
A = Guardar detalle en archivo_detalle.txt
Opcion: S

Tabla de Simbolos -> [NUMERO, RESTA, NUMERO, PARENTESIS_A, NUMERO, IGUAL]
Linea Procesada: 819-664(224= ERROR

-----
Tabla de Simbolos -> [NUMERO, MULTIPLICACION, NUMERO, SUMA, NUMERO, IGUAL]
Linea Procesada: 290*139+539= 40849

-----
Tabla de Simbolos -> [NUMERO, RESTA, NUMERO, MULTIPLICACION, NUMERO, IGUAL]
Linea Procesada: 899-784*737= -576909

-----
--- INICIO DE PROCESAMIENTO CALCULADORA ---

-----
TABLA: OPERACIONES
+      SUMA
-      RESTA
*      MULTIPLICACION

TABLA: SIMBOLOS
(      PARENTESIS_A
)      PARENTESIS_C
=      IGUAL

-----
RESUMEN DE EVALUACIÓN FINAL:
Total de expresiones CORRECTAS (validas): 138744
Total de expresiones INCORRECTAS (invalidas): 246256
```


- **N (Máxima velocidad):** El programa procesará todo sin imprimir detalles en consola, ideal para archivos muy grandes.

```

Seleccione modo de detalle:
S = Mostrar operaciones en consola
N = No mostrar detalle, maxima velocidad
A = Guardar detalle en archivo_detalle.txt
Opcion: n
|--- INICIO DE PROCESAMIENTO CALCULADORA ---
=====

TABLA: OPERACIONES
+      SUMA
-      RESTA
*      MULTIPLICACION

TABLA: SIMBOLOS
(      PARENTESIS_A
)      PARENTESIS_C
=      IGUAL

=====

RESUMEN DE EVALUACIÓN FINAL:
Total de expresiones CORRECTAS (validas): 138744
Total de expresiones INCORRECTAS (invalidas): 246256
Total de operaciones analizadas: 385000
Tiempo transcurrido: 0 minutos y 12 segundos
=====

--- PROCESO FINALIZADO CON ÉXITO ---

```

- **A (Archivo):** Guarda todo el detalle del análisis en un archivo llamado archivo_detalle.txt.

```

<terminated> Analizar [Java Application] C:\Program Files\Java\jre1.8.0_491\bin\javaw.exe (2
Seleccione modo de detalle:
S = Mostrar operaciones en consola
N = No mostrar detalle, maxima velocidad
A = Guardar detalle en archivo_detalle.txt
Opcion: a
|--- INICIO DE PROCESAMIENTO CALCULADORA ---
=====

TABLA: OPERACIONES
+      SUMA
-      RESTA
*      MULTIPLICACION

TABLA: SIMBOLOS
(      PARENTESIS_A
)      PARENTESIS_C
=      IGUAL

=====

RESUMEN DE EVALUACIÓN FINAL:
Total de expresiones CORRECTAS (validas): 138744
Total de expresiones INCORRECTAS (invalidas): 246256
Total de operaciones analizadas: 385000
Tiempo transcurrido: 0 minutos y 4 segundos
Detalle guardado en: archivo_detalle.txt
=====

--- PROCESO FINALIZADO CON ÉXITO ---

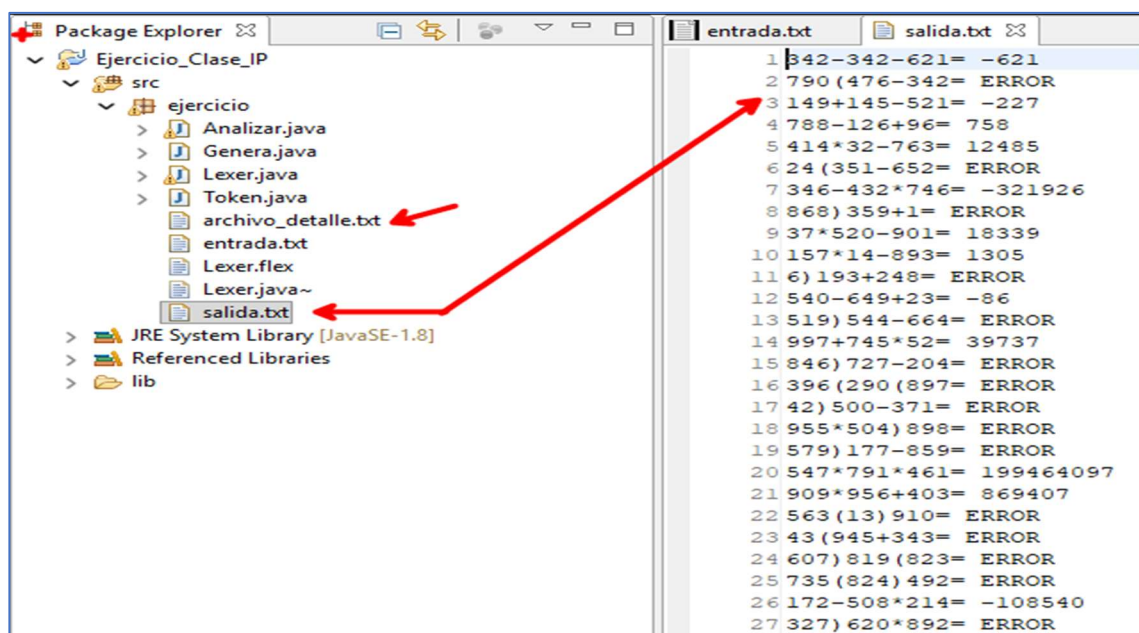
```

4. Obtención de Resultados

Una vez que el proceso finalice, verás un mensaje de "**PROCESO FINALIZADO CON ÉXITO**".

```
=====
RESUMEN DE EVALUACIÓN FINAL:
Total de expresiones CORRECTAS (validas): 138744
Total de expresiones INCORRECTAS (invalidas): 246256
Total de operaciones analizadas: 385000
Tiempo transcurrido: 0 minutos y 4 segundos
Detalle guardado en: archivo_detalle.txt
=====
--- PROCESO FINALIZADO CON ÉXITO ---
```

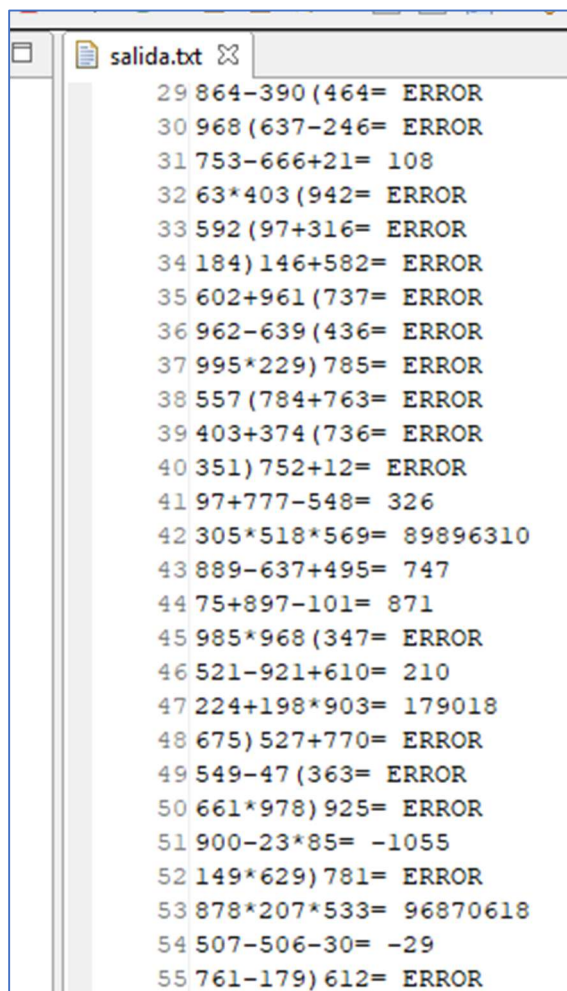
- **Resultados:** Abre el archivo **salida.txt** ubicado en **src/ejercicio/**. Allí encontrarás cada operación seguida de su resultado.
- **Detalles (Si elegiste modo A):** Abre **archivo_detalle.txt** para ver el desglose paso a paso de cómo la calculadora resolvió cada expresión.



5. Resolución de Errores

Si el sistema encuentra un error, no detendrá el proceso, sino que marcará la expresión como **ERROR** en el archivo salida.txt. Las causas comunes son:

- **División por cero.**
- **Paréntesis desbalanceados:** Expresiones con apertura (sin su respectivo cierre).
- **Sintaxis inválida:** Operadores mal posicionados o caracteres no reconocidos (como letras o símbolos fuera del diccionario de Token.java).



```
29 864-390 (464= ERROR
30 968 (637-246= ERROR
31 753-666+21= 108
32 63*403 (942= ERROR
33 592 (97+316= ERROR
34 184) 146+582= ERROR
35 602+961 (737= ERROR
36 962-639 (436= ERROR
37 995*229) 785= ERROR
38 557 (784+763= ERROR
39 403+374 (736= ERROR
40 351) 752+12= ERROR
41 97+777-548= 326
42 305*518*569= 89896310
43 889-637+495= 747
44 75+897-101= 871
45 985*968 (347= ERROR
46 521-921+610= 210
47 224+198*903= 179018
48 675) 527+770= ERROR
49 549-47 (363= ERROR
50 661*978) 925= ERROR
51 900-23*85= -1055
52 149*629) 781= ERROR
53 878*207*533= 96870618
54 507-506-30= -29
55 761-179) 612= ERROR
```